explAIn

How models learn

Weights change only in training. Every other change lives in context, memory or state.

"The model keeps capacity, not records."

Ask on every project

1. Does this need new behavior from the model, or just new facts in the context?
2. How often does this information change, and who must be able to update it?
3. Should this change reach one request, one user, one tenant or everyone?
4. Is this fact frequent, rare, private or newer than the training cutoff?
5. When the provider ships a new version, what do we have to re-test or redo?

Rules of thumb

- Keep facts that change in a source of truth fed into context, never in the weights.
- Consider fine-tuning for consistent style, format or a narrow task, not to add facts.
- Never read a wrong answer to judge whether it knew. Check a source you control.
- Treat user feedback as logged data: it changes nothing until someone runs training.

Red flags

- ⚠ "Users will correct it and it'll learn over time", with no pipeline behind it
- ⚠ A plan to fine-tune on docs so the model "knows" current policies
- ⚠ A feature that treats the model's own "I'm not sure" as a signal
- ⚠ A provider version upgrade with no re-test of prompts, formats and refusals

Key terms

Parameters	The numbers (weights) set by training. They don't move while the model is used.
Post-training	Further training that turns a text predictor into an assistant with set behaviors.
Overfitting	Memorising the training data instead of learning patterns that transfer.
Memory	Stored notes or history that the system re-sends as context on later requests.
Application state	What your product writes (orders, files, logs); it reaches the model only if sent.

DECISION GUIDE

Where does this change live?

If it matters for this request only	→ **put it in the context.**
If one user needs it across sessions	→ **save it as memory, re-sent as context.**
If it's a fact that changes, like a price or policy	→ **fix the source of truth and retrieve it.**
If it records what happened, like an order or feedback	→ **it's application state.**
If you need behavior that prompts can't get	→ **consider a fine-tune, then evaluate it.**
If the provider ships a new version	→ **your parameters moved; re-test prompts and formats.**